第四届 eBPF开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

融合eBPF与AI技术的微架构能效分析研究

小米科技有限责任公司 | 内核技术部

曲盼旺 | 冷万昌 | 吴雨薇

陕西·西安 2026年4月18日

目录

01 传统系统能效优化痛点
02 eBPF + AI 融合的系统架构
03 微架构能效分析系统
04 核心创新技术介绍
05 工程实践 & 能效新纪元

1.0 技术背景：传统系统能效优化痛点

1.采集扰动大

传统工具采集时数据量大、负载高，工具自身的运行即会产生高额能耗，无法胜任实时分析

方法	systrace	simpleperf	eBPF
性能开销	1%~5%	10%~25%	<1%
功耗开销	1%~5%	>15%	<1%

2.微架构归因难

微架构能效影响因子庞杂，难以梳理因果传导链路，极度依赖专家的排查经验

3.缺乏统一标准流程

从数据采集到诊断输出，各环节工具链割裂，缺乏统一的自动化规范流程，容易顾此失彼，返工分析，影响效率

1.1 系统优化目标的转变：从 峰值性能 到 体验优先

基础体验决定长期可用性

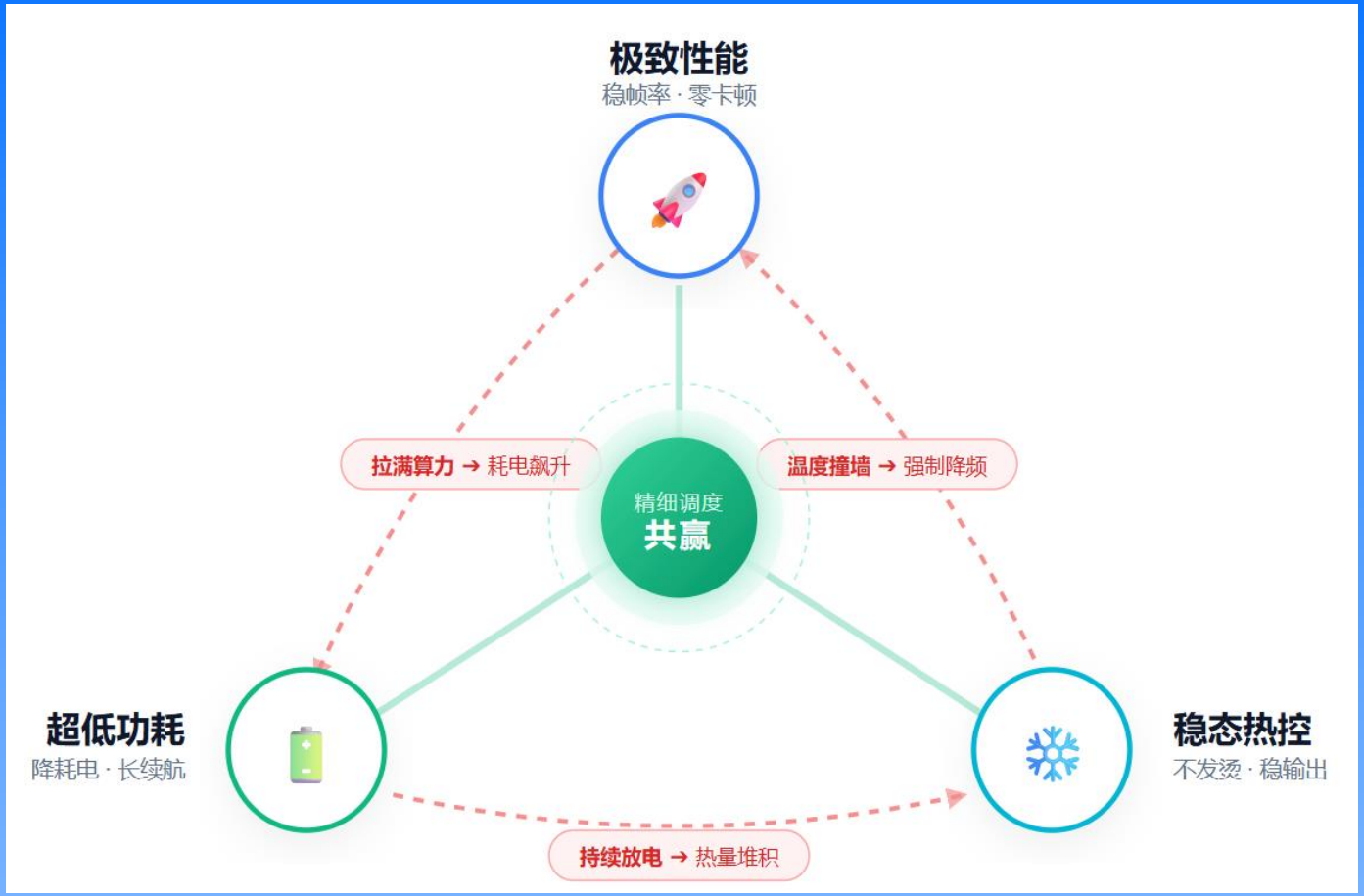
从追求“瞬时峰值”转向“能效优先”，构建性能、功耗与温控的长效稳态平衡。

DVFS与调度策略的影响

动态电压频率调整(DVFS)和任务调度影响帧率、卡顿率及启动时延，决定交互流畅度。

能效分析新范式

需同时刻画功耗与性能的耦合关系，支持面向任务、线程级别的精确定位与优化决策。



2.0 能效分析新范式 eBPF+AI 的三层系统架构

全栈运转流程

底层感知

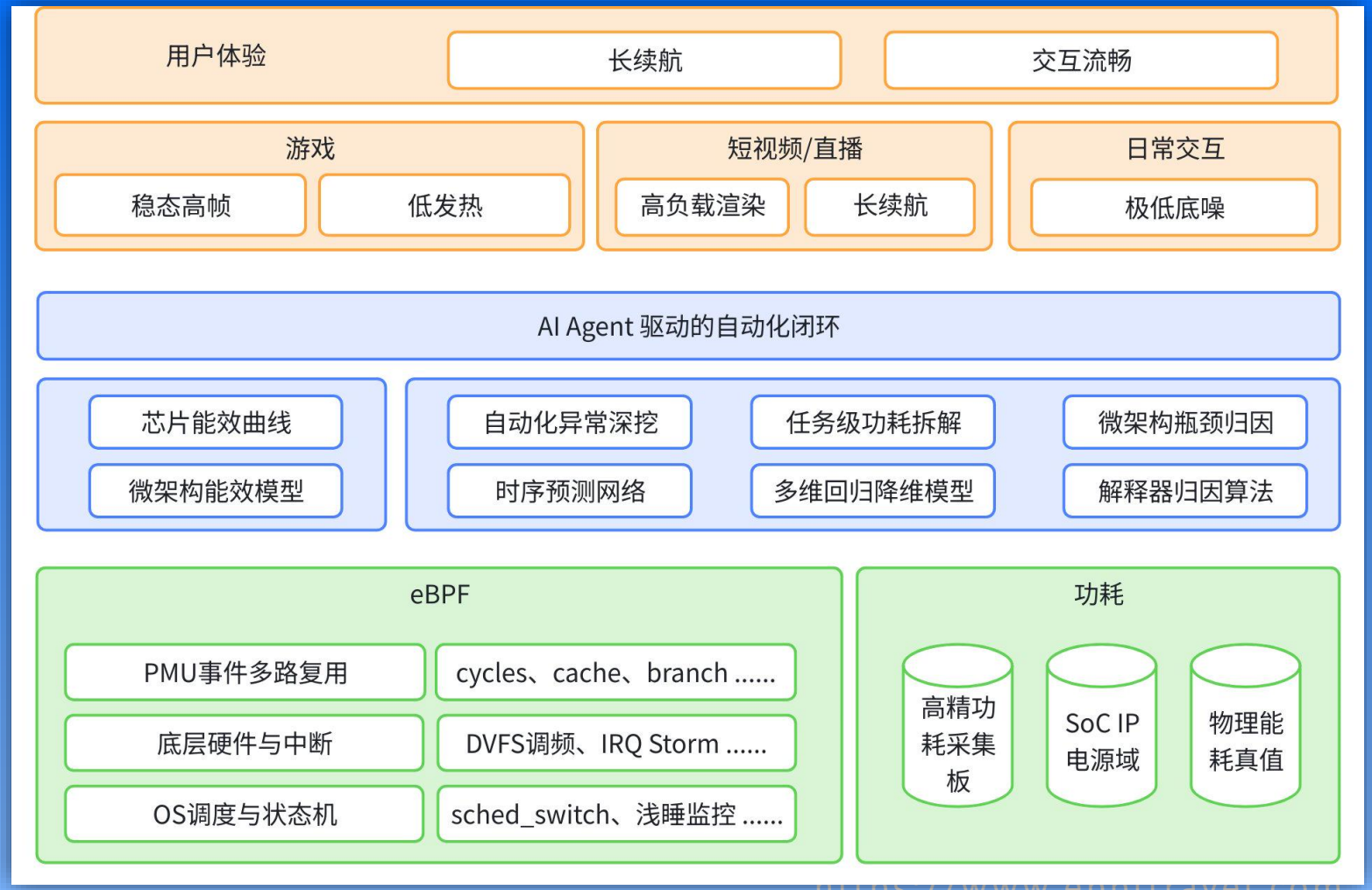
eBPF特征与硬件功耗双向提取

中层 AI诊断

AI跨域降维与物理模型交叉验证

顶层业务执行

结合场景和用户行为多Agent引擎接管并输出修复策略



https://www.ebpftraver.com

2.1 全程机械手参与的Agent自动化诊断流程

整体流程

eBPF 数据采集
Task级 / 内核态 / 硬件PMU

AI 诊断中枢
功耗分配/微架构归因

Agent自动化闭环
流水线编排/策略输出

数据采集与对齐

利用 eBPF 技术采集 Task 级 PMU 事件、内核行为及系统状态特征，并与硬件级功耗数据进行毫秒级时间对齐，确保数据同源性。

能效建模与智能诊断

融合 AI 回归模型与解释器算法，将 Cluster 级功耗拆解至具体 task，深度解析底层数据，诊断 Cache/Branch Miss 等微架构算力空转黑洞

Agent 驱动与全自动分析流程

Agent 架构全程接管排查流程，自动串联各个环节，最终输出代码级优化策略，实现分析流程的绝对标准化和自动化

3.0 AI Agent驱动的微架构能效分析系统

eBPF多源数据采集

kprobe / tracepoint	CPU调度事件 sched_switch, sched_wakeup
perf_event（PMU）	硬件计数器 IPC, Cache Miss, Branch
cgroup / energy	Per-cgroup 能耗归因 进程级功耗分摊
XDP / TC	网络包级功耗 WIFI / 5G Modem

功耗域采集

功耗板 | CPU | ddr | Gpu …

时间对齐引擎 500ms Grid · epoch_s 统一时间轴

多源数据融合

AI多层分析引擎

① Pearson 相关性分析	Cross-domain关联 交叉滞后±6 windows
② 异常热点检测	10+检测规则 SM8850 PMU 基线
③ LLM 根因推理	Claude 4.6 / DeepSeek-R1 RAG + 反模式匹配
④ 决策融合 + Safety	规则+ RAG + ML 投票 SPRT灰度验证

LangGraph StateGraph 编排

Planner | Collector | Analyzer | Decider | Executor | Verifier

HTML报告 因果图谱 知识沉淀 自动调参

3.1 核心技术1：eBPF 低扰动数据采集系统

统一采集能力

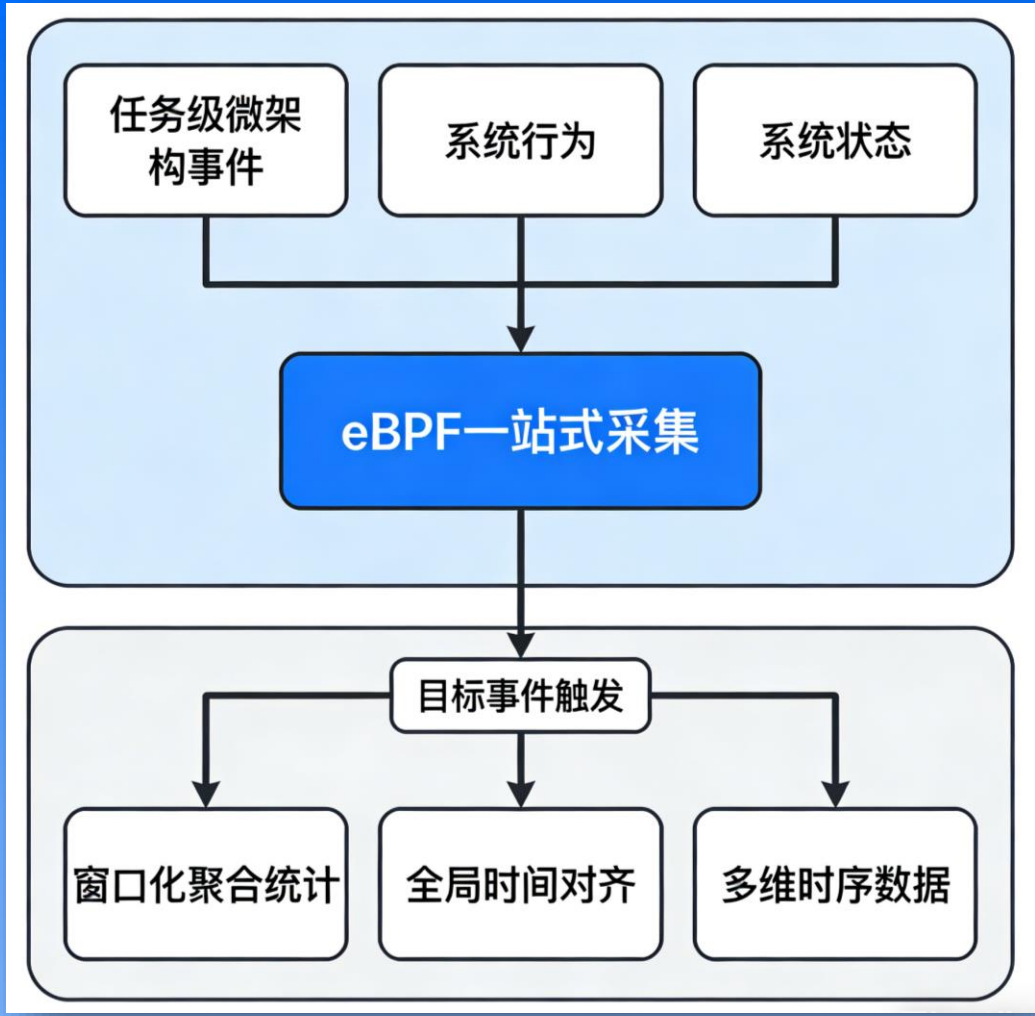
在内核态统一采集任务级微架构事件、系统行为及状态信息，打破数据孤岛，实现全链路监控。

低开销运行

采用事件驱动机制和窗口化聚合统计，避免轮询损耗，将运行时开销降至最低，消除业务干扰。

全局时间对齐

以统一时间窗口组织采集数据，确保多维度数据在时间轴上的一致性，为关联分析提供基础。



3.1.1 eBPF探针实现：并发多维采集架构

Hook 点设计

采集点（Hook）	监控指标与分析逻辑
sched_switch/sched_wakeup	采集上下文切换与大核唤醒事件，定量计算进程唤醒大核比例
cpu_frequency	追踪DVFS调频事件，归因热控驱动vs调度驱动
irq_handler_entry/exit	中断风暴检测（统计rate/s并进行severity分级
cpu_idle	浅睡浪费统计（休眠<100us且残留>500次告警

PMU 硬件事件多路复用

- **轮转窗口**：10ms 高频切换（突破物理计数器限制）
- **四路并发**：时钟周期 / 指令数 / Cache缺失 / 分支缺失
- **结构输出**：直方图 / 计数 / 折叠栈（原生对接 bcc & bpftrace）

asyncio 四路并发采集引擎

- **消除阻塞**：asyncio.gather 四路采集（Ftrace/CPU/GPU/内存）
- **极低开销**：单通道系统扰动 **< 1%**（事件驱动 + 窗口聚合）

SoC 采集规格

```
Hook:  sched_switch
       sched_wakeup
       cpu_frequency
       irq_handler_*

PMU:   cycles/instrs
       cache/branch-miss

Interval: 10ms mux

Async: ftrace+cpu_pmu
       +gpu_pmu+memory

Overhead < 1%
```

3.1.2 数据对齐：统一时间轴+双窗口关联分析

多源时间戳归一化

- cpu_pmu（hperf4mi）：原生 epoch_s；
- powerboard（QEPM）：原生 epoch_s
- ftrace: boot_epoch + boot_time_s 对齐；
- sysfs polling（cpu_freq/gpu/thermal）：epoch_ns/1e9；
- 所有源时钟归一化，同时消除多源时钟漂移，保证时序一致性

双窗口 Pearson 相关性系数 + Lag 时间滞后关联分析

- **500ms 粗窗口**：性能功耗全局Pearson 系数趋势相关性分析
- **50ms 精窗口**： 事件级因果时序精细化分析，捕获调频→功耗→帧率微延迟
- **先验关联过滤**：预过滤 15 类 DVFS 先验耦合对：同模块频率负载耦合对、热耦合、调度耦合等，聚焦具有**高诊断价值**的异常关联，

双窗口设计：

➢ **500ms 粗窗口**

全局功耗/性能趋势
Pearson 相关分析

➢ **50ms 精窗口**

事件级因果时序
详细分析

过滤先验耦合对：

聚焦高价值跨域
异常关联

3.2 核心技术2：AI 自动化处理

能效建模：从cluster到task

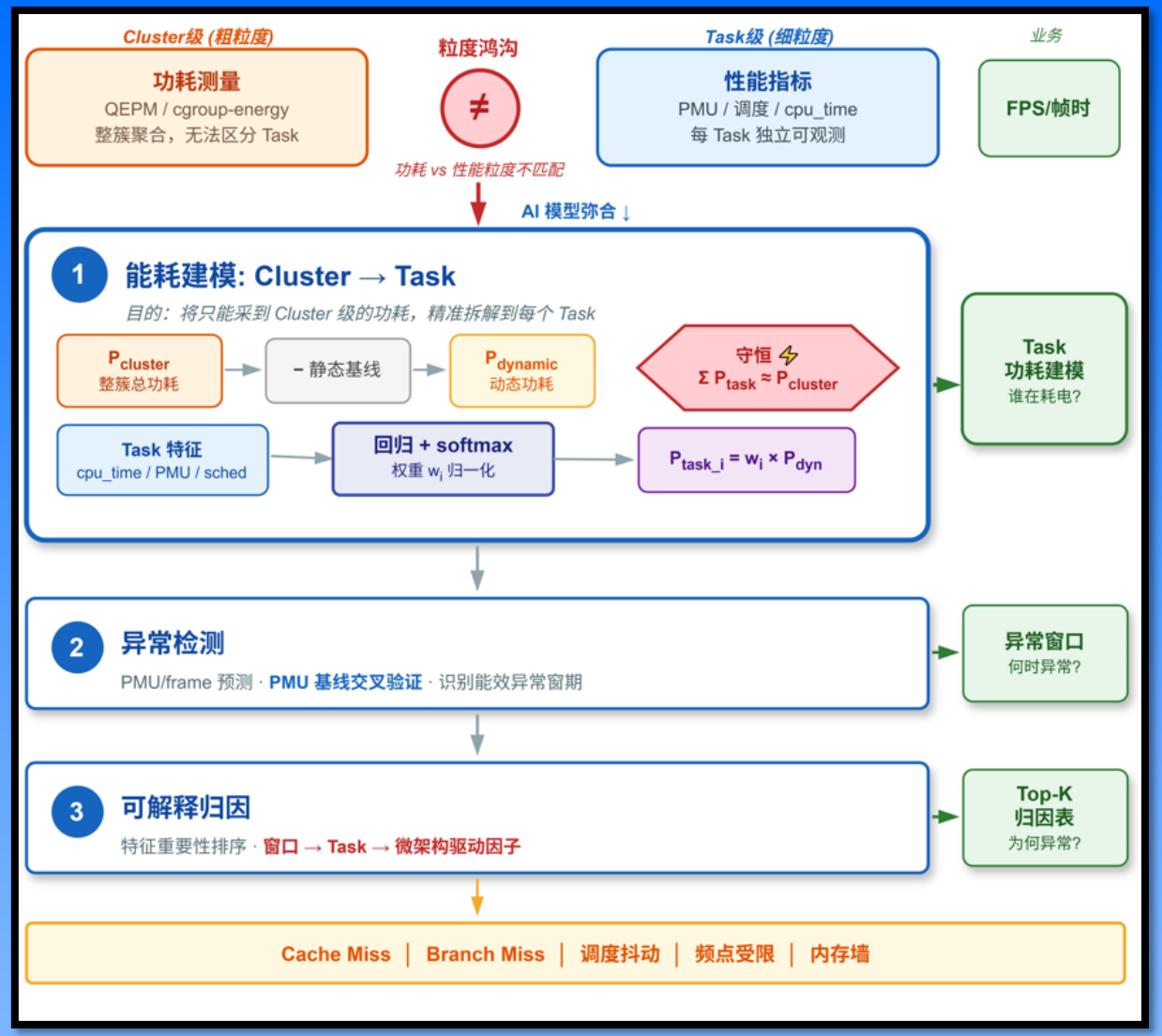
以 QEPM cluster 功耗为监督信号，基于 Task 级 PMU/调度特征训练回归模型，输出层施加 softmax + 窗口能量闭合约束（ΣTask ≈ Cluster动态功耗）

异常预警：异常精准识别

结合IPC、吞吐量等指标建立预测模型，精准识别能效异常窗口，实现问题的即时发现。

可解释归因：定位关键特征

基于特征重要性排序输出 Top-K 驱动因子（Cache Miss / Branch Miss / 调度抖动 / 频点受限），指向代码级优化方向



3.2.1 AI-能效建模

粒度鸿沟

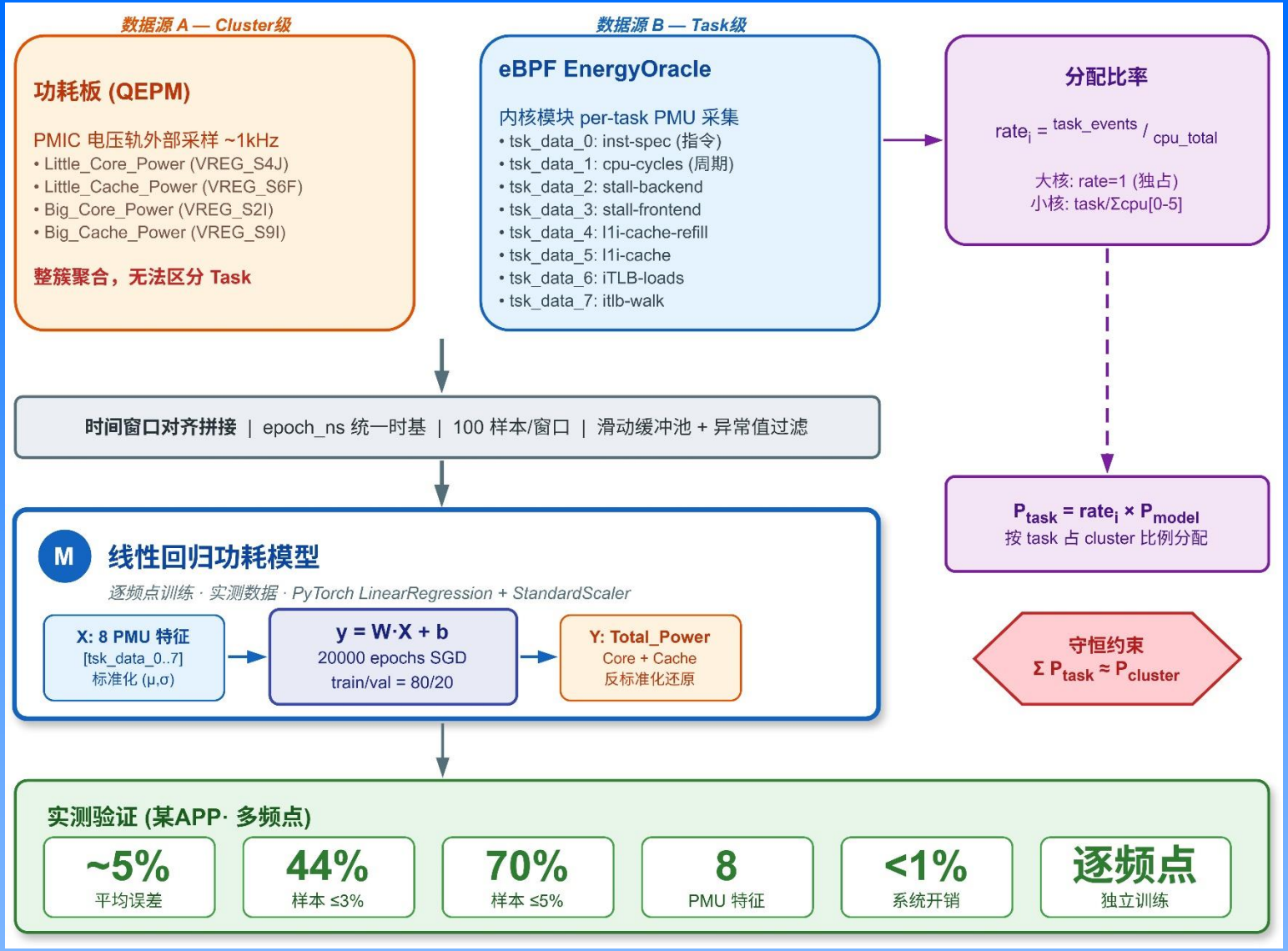
- 功耗采集止于 Cluster 级
- 性能指标精确到 Task 级
- 拟合到 Task 级功耗

建模策略

- 8 个 PMU 特征 → 线性回归
- 逐频点独立训练
- rate = task_events / cpu_total

能量守恒

- Σ P_task ≈ P_cluster（闭合约束）
- 系数 ×10^8 整型化嵌入内核模块
- 端侧实时计算，系统开销 <1%



3.2.2 事件触发深挖：因果链追踪与归因

功耗异常深挖

- **触发条件**：任一电源域 avg_power > 500mW 或功耗热点 severity > 7；
- **Top-5 域：** 按 avg 排序做时序分解识别功耗尖峰窗口（> 2× 均值为 spike），>3 个 spike 产生 power_spike_cluster 新热点（evidence_type: deep_dive）

CPU 饱和 + Jank 帧率抖动深挖

- **CPU饱和触发**：prime_saturated / big_core_waker 热点；分析唤醒树 Top-5 + simpleperf Top-10 函数 + idle_efficiency
- **Jank 触发**：P99 > 2× P50；帧级窗口关联 GPU 负载（>85%）/ CPU 调度抢占 / 内存 IO 阻塞，输出精准归因

瓶颈归因

- **引入解释器算法 (Interpreter) 深度解析硬件事件**：
将能效劣化精准归因为多维微架构瓶颈（诸如 Cache Miss 内存墙阻塞、分支预测，流水线刷新、执行单元停顿等），输出代码级优化提示

性能与功耗双维触发器

- Power 异常
（跨域功耗异常）
- CPU Saturated
（CPU 饱和与算力浪费）
- Jank
（长尾掉帧与体验劣化）

瓶颈归因：解释器算法

https:

4.0 工程实践： 某短视频滑动场景的分析案例

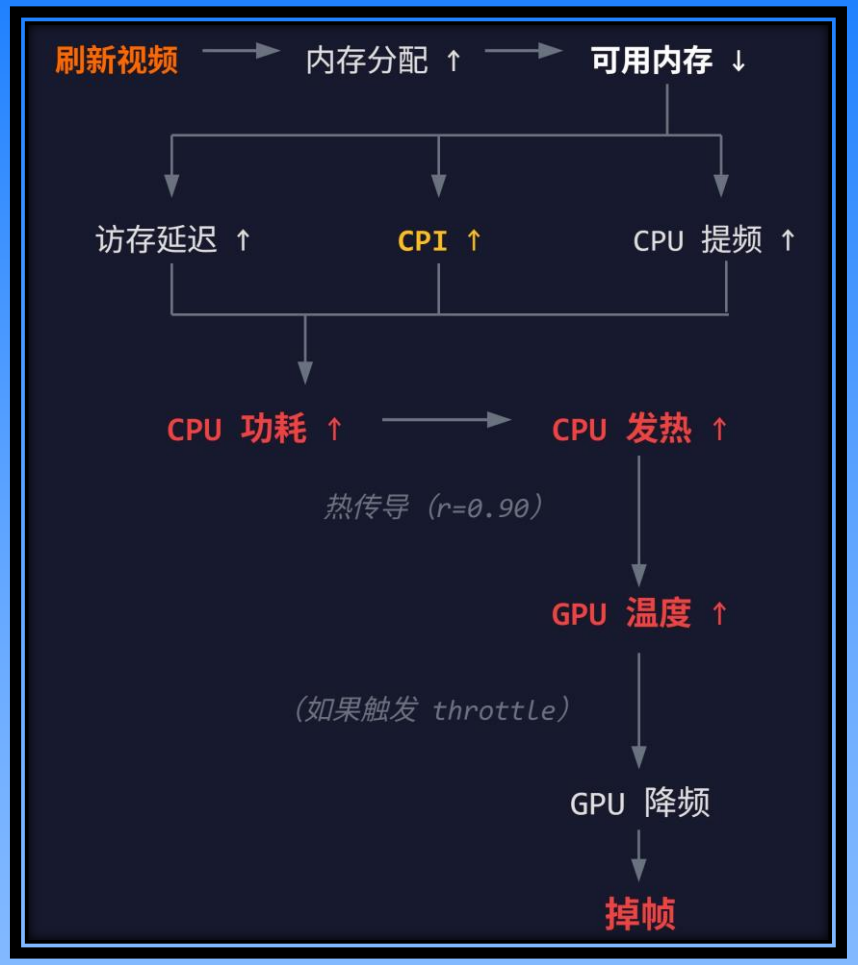
系统指标

CPU大核频率	GPU频率	峰值温度	CPU利用率	可用内存最低	eBPF 功耗开销
1847 MHz	580 MHz	41.2 °C	23.4 %	2847 MB	0.8 %

Pearson 相关性 (Top 5)

Source A	Source B	relation	诊断
cpu_freq_big	thermal_cpu	+0.901	HIGH
mem_alloc_rate	cpu_freq_big	+0.825	HIGH
gpu_busy_pct	thermal_gpu	+0.789	HIGH
cache_miss	mem_alloc	+0.699	HIGH
mem_avail	cpu_freq_big	-0.623	HIGH

AI 因果链推理



Top 5 异常热点归因

- 视频切换内存突发分配
- 大核频率虚高
- 视频解码热传导链
- SF调度抖动(312次迁移
- L2 Cache Miss偏高

优化策略实施

- 调整负载衰减 → CPU功耗-15~20% 优化
- SF固定中核 → 减少迁移开销
- 视频解码预分配buffer → 降低kswapd
- 评估DDR DCVS → 降低访存延迟
- 切换full档位 → 获取完整PMU数据

演进与展望：基于eBPF的 2026 能效自治新纪元

Software & Ecosystem 系统架构与软件生态

Hardware & Macro Trends

01 eBPF 演进：从被动观测到主动干预

➢ 现阶段 · **可观测**

基于 **perf_event + 多维 eBPF Hook**，实现低扰动的任务级功耗与微架构热点剖析

➢ 2026 · **主动干预**

系统防御 TC / LSM Hook

eBPF TC/LSM 探针 + AI 意图识别，智能拦截网络唤醒风暴与高频唤醒锁

可插拔调度 sched_ext (EAS 2.0)

基于 BPF Scheduler 框架，AI 直接下发 BPF 字节码，实现场景级调度策略热插拔

02 eBPF 在功耗跨层协同管理发挥作用

Android 统一标准

推进 Power Efficiency Framework，整合 **碎片化** 功耗管理接口

意图驱动资源分配

应用显式声明：**PowerHint**， 融合 Doze / JobScheduler / Performance Hint API技术接口

端侧大模型功耗挑战

深度量化 Always-On AI Bot 等后台服务的唤醒代价，建立 LLM 推理功耗基准体系

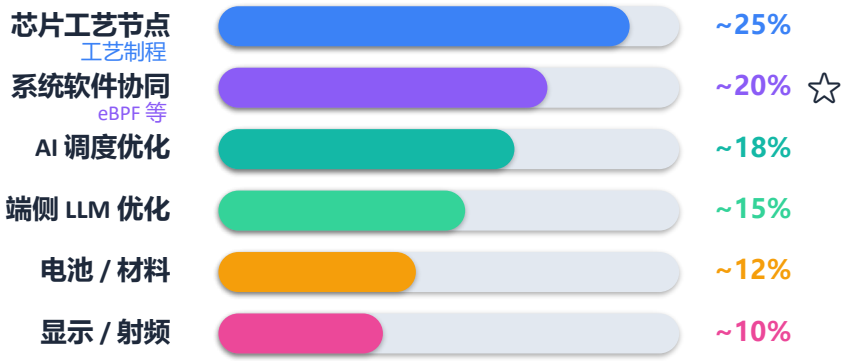
03 通信基建：eBPF 在协议层能效优化

通信技术	核心机制	功耗收益
Wi-Fi 7	320MHz 宽频道，更短活跃期（快传即休眠）	显著降低通信耗电
5G-A	专为 IoT 裁剪的轻量化 5G 协议	连接功耗 -50%
BT 6.0	BLE Channel Sounding 精准测距	极低功耗测距

趋势洞察：新一代通信协议的核心思路一致——**缩短活跃通信时间，更快进入低功耗休眠态**。eBPF TC Hook 可在协议层之上进一步优化唤醒模式

04 全局预判： eBPF 软件定义能效新纪元

旗舰机日常续航综合提升预估：+30% ~ 45%



THANKS

感谢聆听